第8章: データベースとモデル詳細 — データの中身と業務ルール（完全版）

> この章では、本アプリの **心臓部であるデータベース** を、Django（ジャンゴ）の「モデル」という仕組みを通して **1モデル・1フィールドずつ** 読み解きます。さらに、このアプリで最も難解で最も重要な「**1日を288枠（5分刻み）で表す工数データ**」と「**`#HHMMHHMM` という休憩文字列**」の設計を、実際の処理コード（`kosu_utils.py`）の **1行ずつ** に照らして完全に解き明かします。
>
> この章を読み終えると、管理画面（Django Admin）でデータを見たときに「この列は何を表すのか」「なぜこんな形でデータが入っているのか」が全部わかるようになります。バグ調査・データ修正・仕様変更の **すべての出発点** がここにあります。長いですが、辞書のように何度も戻ってきてください。

この章で学ぶこと

- **モデルとは何か** — Pythonのクラスが、なぜデータベースの「表」になるのか
- **フィールド型の全種類** — `IntegerField` `CharField` `BooleanField` `DateField` `ForeignKey` `TextField` `JSONField` などの意味と使い分け
- **本アプリ全10モデルの完全解説** — `member` / `Business_Time_graph` / `team_member` / `kosu_division` / `def_choice` / `administrator_data` / `inquiry_data` / `AsyncTask` / `Operation_history` / `History`
- **ER関係図** — どのモデルがどのモデルとつながっているか
- **【最重要】1日288枠（5分刻み）設計** — `time_work` という1本の文字列で1日の作業をどう表すか
- **【最重要】休憩文字列 `#HHMMHHMM` の読み方** — たった9文字に何が詰まっているか
- **`kosu_utils.py` の実コード逐行解説** — `time_index` / `break_time_process` / `kosu_write` / `break_time_delete` / `break_time_write` / `detail_list_summarize` / `judgement_check`
- **`member` の「直 × 休憩」大量フィールド** — なぜ24個も休憩欄があるのか
- **`shop`（ショップ）選択肢** と **`authority` / `administrator`（権限）** の意味
- **`kosu_division` の最大50区分とVer（バージョン）管理**
- **`AsyncTask` / `History` の `save` 上書きによる件数上限の自動削除**
- **`on_delete=SET_NULL`** の意味と、人員を消したときデータがどうなるか
- **保守の心得** — このデータ構造を壊さないための注意点

目次

1. そもそもモデルとは何か
2. フィールド型の早見表

3. 本アプリのER関係図（全体地図）
4. モデル①: member（人員）
5. モデル②: Business_Time_graph（工数記録）
6. 【最重要】288枠・5分刻み設計の正体
7. 【最重要】休憩文字列 #HHMMHHMM の読み方
8. kosu_utils.py 逐行解説①: time_index
9. kosu_utils.py 逐行解説②: break_time_process
10. kosu_utils.py 逐行解説③: kosu_write
11. kosu_utils.py 逐行解説④: break_time_delete
12. kosu_utils.py 逐行解説⑤: break_time_write
13. kosu_utils.py 逐行解説⑥: detail_list_summarize
14. kosu_utils.py 逐行解説⑦: judgement_check（工数整合性判定）
15. モデル③: team_member（チーム）
16. モデル④: kosu_division（工数区分定義・最大50区分）
17. モデル⑤: def_choice（作業詳細の選択肢）
18. モデル⑥: administrator_data（システム設定）
19. モデル⑦: inquiry_data（問い合わせ）
20. モデル⑧⑨⑩: AsyncTask / Operation_history / History
21. 保守の心得（このデータを壊さないために）
22. トラブルシューティング
23. 演習問題
24. この章のまとめ

---

1. そもそもモデルとは何か

> **モデル（model）とは？** Djangoで「**データベースの表（テーブル）を、Pythonのクラスとして表したもの**」です。プログラマはSQL（エスキューエル：データベースを操作する専用言語）を書かなくても、Pythonのクラスを書くだけで表を作れます。

1.1 たとえ話：モデルは「表の設計図」

データベースを **Excelの表** だと思ってください。

id	従業員番号	氏名	ショップ
1	1001	田中	P
2	1002	佐藤	R

- **1行 = 1件のデータ**（=「レコード」「インスタンス」）
- **1列 = 1つの項目**（=「フィールド」「カラム」）
- **列の名前と型（数字か文字か）を決めた紙** =「モデル」

つまりモデルは、Excelで言う「**1行目の見出しと、各列に何を入れるか決めたルール**」です。本アプリの `models.py` は、この設計図を10枚集めたファイルです。

> **用語まとめ（よみがな：意味）** - **テーブル（table：表）** … データを格納する1つの表。- **レコード（record：記録）** … 表の1行。1件分のデータ。- **フィールド（field：項目）** … 表の1列。`employee_no` など。- **インスタンス（instance：実体）** … Pythonでモデルから作った1件のデータ。レコードとほぼ同義。- **マイグレーション（migration：移行）** … モデルの変更を実際のデータベースに反映する作業。`python manage.py makemigrations` → `migrate` で行う。

1.2 モデルの最小の形（※説明用の簡易例）

> **▼ このコードがやること（先に日本語で）**：「人員」という表を作り、「従業員番号（数字）」「氏名（最大100文字）」という2つの列を定義する、いちばん簡単な例です。これは本物のコードではなく、形を理解するための簡易例です。

コピー

```
# ※説明用の簡易例（本アプリの実コードではありません）
# Djangoのモデル機能を読み込む
from django.db import models

# models.Model を継承すると「表」になる
class Member(models.Model):
    # 整数の列
    employee_no = models.IntegerField('従業員番号')
    # 最大100文字の文字列の列
    name = models.CharField('氏名', max_length=100)
```

- `class Member(models.Model):` … `models.Model` を **継承（けいしょう：親クラスの機能を引き継ぐこと）** すると、このクラスが自動的にデータベースの表になります。
- `models.IntegerField(...)` … この列には **整数（Integer）** が入る、という宣言。
- `models.CharField(..., max_length=100)` … この列には **文字列（Character）** が入る。`max_length` は最大文字数で、`CharField` では必須です。

> **なぜ？** クラスにフィールドを書くだけで表が作れるのは、Djangoが裏で「このクラスをSQLのCREATE TABLE文に翻訳してくれる」からです。私たちはPythonの言葉だけで会話でき、データベースの方言（SQL）を覚えなくて済みます。

1.3 `__str__` メソッドの役割

本アプリの全モデルには、最後に `def __str__(self):` という関数が付いています。

> **▼ このコードがやること（先に日本語で）**：1件のデータを「人間が読める文字」で表す関数です。管理画面（Django Admin）で一覧表示するときに、ここで返した文字が見出しとして表示されます。

コピー

```
# このデータを文字にするとき呼ばれる
def __str__(self):
    # 氏名を返す（管理画面に氏名が表示される）
    return self.name
```

- `self` … 「その1件のデータ自身」を指す決まり文句。
- `return self.name` … その人の氏名を返す。これがないと、管理画面で `member object (1)` のような無味乾燥な表示になってしまいます。

---

2. フィールド型の早見表

本アプリで登場するフィールド型を、最初にまとめて覚えておきましょう。以降の解説がぐっと楽になります。

型	よみ	入るもの	たとえ	本アプリの例
`IntegerField`	インテジャーフィールド	整数	個数・番号	`employee_no`（従業員番号）
`CharField`	キャラフィールド	短い文字列（`max_length` 必須）	名前・記号	`name`（氏名）、`time_work`（作業内容）
`TextField`	テキストフィールド	長い文字列（長さ無制限）	文章・説明	`inquiry`（問い合わせ本文）、各定義
`BooleanField`	ブーリアンフィールド	True / False（真偽）	はい/いいえ、ON/OFF	`authority`（権限）、`follow`
`DateField`	デイトフィールド	日付	カレンダーの1日	`work_day2`（就業日）
`DateTimeField`	デイトタイムフィールド	日付＋時刻	タイムスタンプ	`created_at`（作成日時）
`ForeignKey`	フォーリンキー	他の表への参照	別表へのリンク	`name`（→ member）
`JSONField`	ジェイソンフィールド	JSON（構造化データ）	入れ子のメモ	`changes`（変更内容）

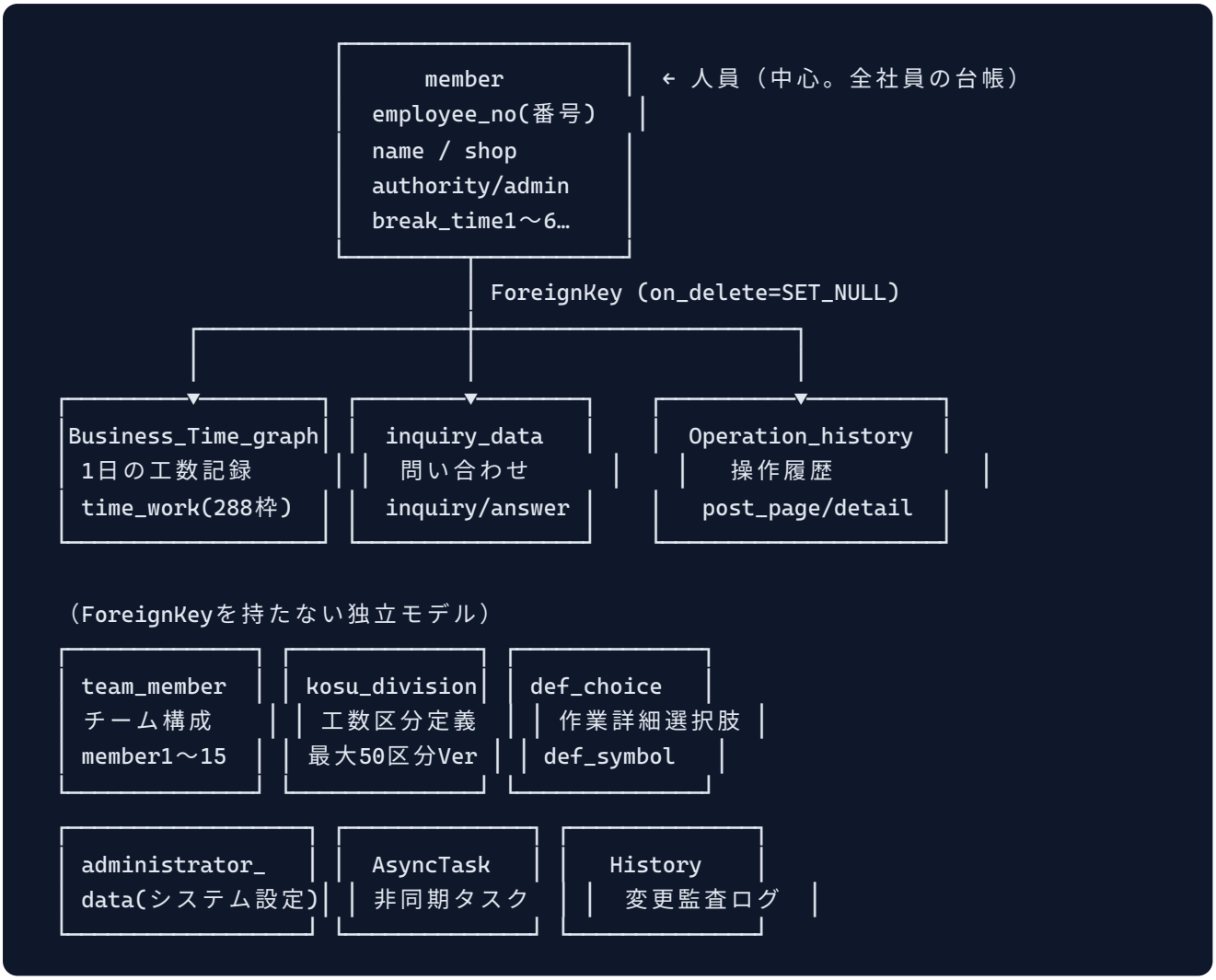
> **用語: `null` と `blank` の違い** ⚠️ よく混同するので注意。- `null=True` … **データベース上** で「空（NULL）」を許す。（数字や日付の「未入力」を表せる） - `blank=True` … **入力フォーム上** で「空のまま送信」を許す。（管理画面で必須にしない） 多くのフィールドは両方付けますが、`CharField` は慣習的に `null=True` を付けないこともあります（空文字 `''` で代用できるため）。本アプリでは両方付けている箇所が多いです。
>
> **用語: `verbose_name`（バーボスネーム：表示名）** … `models.IntegerField('従業員番号')` の `'従業員番号'` の部分。管理画面に表示される日本語ラベルです。プログラム内部の名前（`employee_no`）とは別物で、人間向けの「見出し」です。

---

3. 本アプリのER関係図（全体地図）

> **ER図（イーアールず：Entity-Relationship Diagram）とは？**「どの表（エンティティ）が、どの表とつながっているか（リレーションシップ）」を描いた地図です。本アプリの10モデルの関係は次の通りです。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー



ポイント： - **`member` が中心。**多くの表が `ForeignKey` で `member` を指しています。- `Business_Time_graph`・`inquiry_data`・`Operation_history` の3つは、`name` フィールドで `member` を参照しています（だれの記録か、を結びつける）。- それ以外（`team_member`・`kosu_division`・`def_choice`・`administrator_data`・`AsyncTask`・`History`）は、`member` への直接の `ForeignKey` を持たない独立した表です（従業員番号を文字列で持つだけ等）。

---

4. モデル①: member（人員）

これは本アプリで最も重要な、**全社員の台帳** です。`models.py` の冒頭から見ていきます。

4.1 shop_list（ショップの選択肢）

> **▼ このコードがやること（先に日本語で）:**「ショップ（その人が所属する勤務区分）」として選べる値のリストを作ります。プルダウンメニューの選択肢になります。

コピー

```
# 「人員」の表を定義
class member(models.Model):
  # ショップの選択肢リスト（左:DB保存値, 右:表示文字）
  shop_list = [
    # ショップP
    ('P', 'P'),
    # ショップR
    ('R', 'R'),
    # ショップW1
    ('W1', 'W1'),
    # ショップW2
    ('W2', 'W2'),
    # ショップT1
    ('T1', 'T1'),
    # ショップT2
    ('T2', 'T2'),
    # ショップA1
    ('A1', 'A1'),
    # ショップA2
    ('A2', 'A2'),
    # ショップJ
    ('J', 'J'),
    # 上記以外
    ('その他', 'その他'),
    # 役職者（B番系）
    ('組長以上(P,R,T,その他)', '組長以上(P,R,T,その他)'),
    # 役職者（C番系）
    ('組長以上(W,A)', '組長以上(W,A)'),
    # 在籍していない人（一覧で後ろに回す扱い）
    ('異動・退社', '異動・退社'),
  ]
```

- `shop_list = [ ... ]` … タプル（2つ組）のリスト。各タプルは `(保存値, 表示値)` の形。
- 1要素 `('P', 'P')` … 左がデータベースに保存される実体、右が画面に表示される文字。本アプリでは両方同じです。
- `'異動・退社'` … この値の人は、もう普通に働いていない人。最近の改修で「人員一覧の後ろに回す」扱いになりました（git履歴の `人員一覧で異動・退社を後ろに変更` がこれ）。

なぜ？ 選択肢を「リスト」で持つのか: あとで `shop = models.CharField(..., choices=shop_list, ...)` のように `choices` に渡すと、Django管理画面が自動でプルダウンを作り、リスト外の値を弾いてくれます。タイプミスでヘンな値が入るのを防ぐ「ガードレール」です。

⚠️ **超重要:** この `shop` の値は、`kosu_utils.py` の `judgement_check`（§14）や `kosu_sort` の中で、勤務時間帯や年休の整合性チェックの条件に直結しています。**ショップ名を1文字でも書き換えると、工数判定ロジックが壊れます。** 安易に文字列を変えてはいけません。

4.2 基本情報フィールド

▼ このコードがやること（先に日本語で）: 1人の社員の「番号・氏名・所属・権限」を定義します。

コピー

```
# 従業員番号（整数）
employee_no = models.IntegerField('従業員番号')
# 氏名（最大100文字）
name = models.CharField('氏名', max_length=100)
# 所属（§4.1の選択肢から）
shop = models.CharField('ショップ', choices = shop_list, max_length=15)
# 一般権限フラグ（True/False）
authority = models.BooleanField('権限')
# 管理者フラグ（True/False）
administrator = models.BooleanField('管理者')
```

- `employee_no` … 従業員番号。**整数**。ログイン時のID代わりになります。
- `name` … 氏名。最大100文字。
- `shop` … 所属ショップ。`choices=shop_list` でプルダウン化、`max_length=15` で最大15文字（`'組長以上(P,R,T,その他)'` が入るので長め）。
- `authority` … **権限フラグ**。`True` なら「他人の工数も見られる・チーム管理できる」等の一般的な拡張権限を持つ人（組長クラス）。`False` なら自分の分だけ。
- `administrator` … **管理者フラグ**。`True` なら「人員登録・工数区分定義・システム設定など、管理系画面に入れる人」。

用語: フラグ（flag：旗） … `True` / `False` の2択で「その性質を持つか」を表す値。旗が立っていれば（True）その権限あり、と考えると分かりやすいです。

なぜ権限が2段階（authority と administrator）あるのか: 一般社員 → `authority=False, administrator=False`、組長 → `authority=True`、システム管理者 → `administrator=True`、というように **3段階の見える範囲・できることの違い** を表現するためです。フロント側（React）はこの2つのフラグを見て、表示するメニューや遷移できる画面を切り替えています。

4.3 休憩時間フィールド（直 × 種類の大量フィールド）

ここが `member` の特徴的な部分。**24個の休憩フィールド** が並びます。

> **▼ このコードがやること（先に日本語で）**：「勤務形態（直）ごと × 昼休憩・残業休憩1〜3」の組み合わせで、その人の休憩時間帯を全部登録できるようにしています。各値は `#HHMMHHMM` 形式の9文字文字列（§7で詳説）です。

コピー

```
# 1直の昼休憩
break_time1 = models.CharField('1直昼休憩時間', max_length=9)
# 1直の残業休憩1
break_time1_over1 = models.CharField('1直残業休憩時間1', max_length=9)
# 1直の残業休憩2
break_time1_over2 = models.CharField('1直残業休憩時間2', max_length=9)
# 1直の残業休憩3
break_time1_over3 = models.CharField('1直残業休憩時間3', max_length=9)
# 2直の昼休憩
break_time2 = models.CharField('2直昼休憩時間', max_length=9)
# 2直の残業休憩1
break_time2_over1 = models.CharField('2直残業休憩時間1', max_length=9)
# 2直の残業休憩2
break_time2_over2 = models.CharField('2直残業休憩時間2', max_length=9)
# 2直の残業休憩3
break_time2_over3 = models.CharField('2直残業休憩時間3', max_length=9)
# 3直の昼休憩
break_time3 = models.CharField('3直昼休憩時間', max_length=9)
# 3直の残業休憩1
break_time3_over1 = models.CharField('3直残業休憩時間1', max_length=9)
# 3直の残業休憩2
break_time3_over2 = models.CharField('3直残業休憩時間2', max_length=9)
# 3直の残業休憩3
break_time3_over3 = models.CharField('3直残業休憩時間3', max_length=9)
# 常昼の昼休憩
break_time4 = models.CharField('常昼昼休憩時間', max_length=9)
# 常昼の残業休憩1
break_time4_over1 = models.CharField('常昼残業休憩時間1', max_length=9)
# 常昼の残業休憩2
break_time4_over2 = models.CharField('常昼残業休憩時間2', max_length=9)
# 常昼の残業休憩3
break_time4_over3 = models.CharField('常昼残業休憩時間3', max_length=9)
# 連1直の昼休憩
break_time5 = models.CharField('連1直昼休憩時間', max_length=9)
break_time5_over1 = models.CharField('連1直残業休憩時間1', max_length=9)# 連1直の残業休憩1
break_time5_over2 = models.CharField('連1直残業休憩時間2', max_length=9)# 連1直の残業休憩2
break_time5_over3 = models.CharField('連1直残業休憩時間3', max_length=9)# 連1直の残業休憩3
# 連2直の昼休憩
break_time6 = models.CharField('連2直昼休憩時間', max_length=9)
break_time6_over1 = models.CharField('連2直残業休憩時間1', max_length=9)# 連2直の残業休憩1
break_time6_over2 = models.CharField('連2直残業休憩時間2', max_length=9)# 連2直の残業休憩2
break_time6_over3 = models.CharField('連2直残業休憩時間3', max_length=9)# 連2直の残業休憩3
```

数えると、**6種類の直 × 4種類（昼＋残業3） = 24フィールド** です。「直」の番号と意味の対応を表にします。

フィールド接頭	番号	意味（直）	説明
`break_time1`	1直	1直（朝〜）	早番
`break_time2`	2直	2直（昼〜）	中番
`break_time3`	3直	3直（夜〜）	夜勤
`break_time4`	常昼	常昼	日勤固定（`tyoku=='4'`）
`break_time5`	連1直	連続1直	連操の1直（`tyoku=='5'`）
`break_time6`	連2直	連続2直	連操の2直（`tyoku=='6'`）

> **なぜ24個も？（たとえ話）** 想像してください。あなたは交替勤務の工場で働いていて、「朝番のときの昼休みは12:00-12:45」「夜番のときの昼休み（実際は深夜）は別の時間」というように、**勤務時間帯（直）が変わると休憩の時間帯も変わる** のです。さらに残業をすると追加の休憩が最大3回入る。だから「6つの勤務パターン × （昼＋残業3）」を全部登録できるよう、24個のマス目を用意してあるわけです。
>
> ⚠️ **`max_length=9` の意味:** 値は必ず9文字。なぜ9かというと `#HHMMHHMM`（先頭の `#` ＋ 開始時刻4桁＋終了時刻4桁）だからです。§7で1文字ずつ分解します。
>
> **なぜ「直ごとにフィールドを分ける」設計なのか（保守の観点）:** 別表に切り出して正規化する設計もあり得ましたが、このアプリは「1人 = 1行」で全休憩を持つ非正規化を選んでいます。利点は「人員1件を取れば全休憩がそろう（JOINが要らない＝速い・単純）」こと。欠点は「フィールドが多くて目がチカチカする」こと。後任者は **この24フィールドの並びを崩さないこと**（`break_get` 関数がこの命名規則で `tyoku` の値からフィールド名を選んでいる）を守ってください。

4.4 break_get が直からフィールドを選ぶしくみ（24フィールドの使われ方）

24個もある休憩フィールドが、実際にどう使われるのかを `kosu_utils.py` の `break_get` で確認します。

> **▼ このコードがやること（先に日本語で）:** 従業員番号で人員を1件取り、`tyoku`（直）の番号に応じて、その直に対応する4つの休憩フィールド（昼＋残業3）を取り出して返します。

コピー

```
# 休憩時間取得関数
# 引数: 直, ログイン者の従業員番号
def break_get(tyoku, login_no):
  # 休憩時間取得
  # 人員を1件取得
  break_time_obj = member.objects.get(employee_no=login_no)

  # 1直の場合の休憩時間取得
  # 直が1なら
  if tyoku == '1':
    # 1直の昼休憩
    breaktime = break_time_obj.break_time1
    # 1直の残業休憩1
    breaktime_over1 = break_time_obj.break_time1_over1
    # 1直の残業休憩2
    breaktime_over2 = break_time_obj.break_time1_over2
    # 1直の残業休憩3
    breaktime_over3 = break_time_obj.break_time1_over3
  # （2直～連2直も、同じ形で break_time2～break_time6 を選ぶ）
  ...
  # 4つの休憩を返す
  return breaktime, breaktime_over1, breaktime_over2, breaktime_over3
```

- `member.objects.get(employee_no=login_no)` … 従業員番号で人員を1件取得（§16.3で出てくるORM）。
- `if tyoku == '1':` … 直の番号で分岐し、その直の休憩4つを選ぶ。2直なら `break_time2系`、…、連2直なら `break_time6系`。
- **24フィールドのうち、その日の直に合う4つだけが選ばれる** のがポイント。これが「直ごとに休憩を分けて持つ」設計の使いどころです。

4.5 ポップアップ通知フィールド

> **▼ このコードがやること（先に日本語で）**：その人に出すお知らせ（ポップアップ）を最大5件まで持てるようにします。本文とID（既読管理用の識別子）のペアです。

コピー

```
# お知らせ本文1
pop_up1 = models.CharField('ポップアップ1', max_length=255, null=True, blank=True)
# お知らせID1
pop_up_id1 =  models.CharField('ポップアップID1', max_length=255, null=True, blank=True)
# お知らせ本文2
pop_up2 = models.CharField('ポップアップ2', max_length=255, null=True, blank=True)
# お知らせID2
pop_up_id2 =  models.CharField('ポップアップID2', max_length=255, null=True, blank=True)
# お知らせ本文3
pop_up3 = models.CharField('ポップアップ3', max_length=255, null=True, blank=True)
# お知らせID3
pop_up_id3 =  models.CharField('ポップアップID3', max_length=255, null=True, blank=True)
# お知らせ本文4
pop_up4 = models.CharField('ポップアップ4', max_length=255, null=True, blank=True)
# お知らせID4
pop_up_id4 =  models.CharField('ポップアップID4', max_length=255, null=True, blank=True)
# お知らせ本文5
pop_up5 = models.CharField('ポップアップ5', max_length=255, null=True, blank=True)
# お知らせID5
pop_up_id5 =  models.CharField('ポップアップID5', max_length=255, null=True, blank=True)
```

- `null=True, blank=True` … 通知がない人は空のままでOK、という意味。
- 本文（`pop_up1`）とID（`pop_up_id1`）がペア。IDは「この通知をもう読んだか」を見分ける目印として使われます（同じIDの通知を二重に出さない等）。直近コミット `通知誤削除修正` (4d2469a) はこの通知まわりの修正です。

4.6 break_check（休憩エラー有効チェック）

> **▼ このコードがやること（先に日本語で）**：「休憩時間帯に工数を入力したらエラーにするか？」のスイッチです。
> `kosu_utils.py` の `break_time_delete`（§11）の分岐に直結します。

コピー

```
# 休憩枠への工数入力をエラーにするか
break_check = models.BooleanField('休憩エラー有効チェック', null=True)

# このデータを文字にするとき
def __str__(self):
  # 氏名を返す（管理画面に氏名が出る）
  return self.name
```

- `break_check == True` の人 … 休憩枠に工数を入れようとすると **エラーで弾く**（厳格モード）。
- `break_check == False`（または `None`）の人 … 休憩枠の工数を黙って消す（寛容モード）。

§11で、この値が処理をどう分岐させるか実コードで確認します。

5. モデル②: Business_Time_graph（工数記録）

1人 × 1日 × 1直 ＝ 1レコード という、工数の本体データです。「だれが、いつ、どんな作業を、何分やったか」が、ここに全部入ります。

> **▼ このコードがやること（先に日本語で）**：1日分の工数記録の列を定義します。`time_work` という1本の文字列に「1日の作業内容（288枠）」が詰め込まれているのが最大の特徴です。

コピー

```
# 「工数記録」の表
class Business_Time_graph(models.Model):
  # 従業員番号（整数。だれの記録か）
  employee_no3 = models.IntegerField('従業員番号')
  # member表への参照
  name = models.ForeignKey(member, verbose_name='氏名', null=True, on_delete=models.SET_NULL)
  # 使った定義のバージョン名
  def_ver2 = models.CharField('工数区分定義Ver', max_length=100, blank=True, null=True)
  # 就業日（日付）
  work_day2 = models.DateField('就業日')
  # 直（'1'〜'6'）
  tyoku2 = models.CharField('直', max_length=2, blank=True, null=True)
  # ★1日288枠の作業内容
  time_work = models.CharField('作業内容', max_length=288, blank=True, null=True)
  # ★枠ごとの作業詳細
  detail_work = models.CharField('作業詳細', max_length=32676, blank=True, null=True)
  # 残業時間（分）
  over_time = models.IntegerField('残業時間', blank=True, null=True)
  # 適用された昼休憩 #HHMMHHMM
  breaktime = models.CharField('昼休憩時間', max_length=9, blank=True, null=True)
  # 残業休憩1
  breaktime_over1 = models.CharField('残業休憩時間1', max_length=9, blank=True, null=True)
  # 残業休憩2
  breaktime_over2 = models.CharField('残業休憩時間2', max_length=9, blank=True, null=True)
  # 残業休憩3
  breaktime_over3 = models.CharField('残業休憩時間3', max_length=9, blank=True, null=True)
  # 出勤/休出/年休 等
  work_time = models.CharField('就業形態', max_length=100, blank=True, null=True)
  # 整合性OKか（§14で判定）
  judgement = models.BooleanField('工数入力OK_NG', null=True)
  # 休憩を変更したか
  break_change = models.BooleanField('休憩変更チェック', null=True)

  # このデータを文字にするとき
  def __str__(self):
    # 「id__日付:番号」
    return str(self.id) + '__' + str(self.work_day2) + ':' + str(self.employee_no3)
```

1フィールドずつ意味を確認します。

- `employee_no3` … 従業員番号。`member` の `employee_no` と同じ番号を整数で持ちます。`name`（ForeignKey）が `SET_NULL` で消えても番号は残るので、「だれの記録だったか」を後から追える保険になります。
- `name = models.ForeignKey(member, ...)` … **`member` 表への参照（リンク）**。この記録の持ち主の人員データを直接たどれます。`on_delete=models.SET_NULL`（§5.1で詳説）と `null=True` がセット。
- `def_ver2` … この日の工数入力に使った **工数区分定義のバージョン名**（`kosu_division.kosu_name`）。後で「Aは何の作業だったか」を引くために、当時の定義名を記録しておきます。
- `work_day2` … 就業日。`DateField` なので「2026-06-01」のような日付。
- `tyoku2` … 直。`'1'` 〜 `'6'` の文字列（最大2文字）。1〜3直・常昼(4)・連1(5)・連2(6)。`kosu_utils.py` のあちこちで分岐条件になります。
- **`time_work`** … ★この章の主役。`max_length=288` の文字列で、**1日（24時間）を5分刻みの288枠に分け、各枠の作業内容を1文字ずつ並べたもの**。§6で完全解説します。
- **`detail_work`** … 各枠の「作業詳細」を `$` 区切りでつないだ長い文字列。`max_length=32676`（とても長い）。§13で組み立て方を見ます。
- `over_time` … 残業時間（分単位の整数）。`judgement_check`（§14）で工数合計との整合性に使います。
- `breaktime` 〜 `breaktime_over3` … この日に適用された休憩時間（`member` から取ってきた値のコピー）。`#HHMMHHMM` 形式の9文字。
- `work_time` … 就業形態。`'出勤'` `'休出'` `'年休'` `'半前年休'` 等の文字列。`judgement_check` の判定の入口です。
- `judgement` … この記録が整合性チェックを通ったか（True=OK）。`judgement_check` の戻り値が入ります。
- `break_change` … この日、標準の休憩から変更したか。直近のコミット `休憩変更時の工数消去バグ修正`(9156f15) が関係する重要フラグです。

5.1 `on_delete=SET_NULL` の意味（最重要の業務ルール）

> **▼ このコードがやること（先に日本語で）**：「人員（member）が削除されたとき、その人を指していた工数記録の `name` を空（NULL）にする」というルールです。記録自体は消しません。

コピー

```
name = models.ForeignKey(member, verbose_name='氏名', null=True, on_delete=models.SET_NULL)
```

`on_delete` は「参照先（member）が消えたとき、この記録をどうするか」のルールです。選べる主な動作：

設定	意味	このアプリでの是非
`CASCADE`	親が消えたら **この記録も道連れで削除**	✗ 過去の工数が消える（危険）
`SET_NULL`	親が消えたら **`name` を NULL にして記録は残す**	◎ 採用。記録を守れる
`PROTECT`	参照中は親を削除させない	△ 退職者を消せなくなる

本アプリは **`SET_NULL`** を採用。理由は明確で、「**人が辞めても、その人が過去に入力した工数の記録は会社の集計に必要だから消してはならない**」からです。

> **なぜ `employee_no3`（番号）を別に持つのか:** `name`（ForeignKey）が `NULL` になると「だれの記録か」が分からなくなりそうですが、`employee_no3` に従業員番号が **整数のコピーとして残る** ので、退職者の記録も番号で追跡できます。これは意図的な冗長設計（保険）です。
>
> ⚠️ **保守の注意:** `inquiry_data` と `Operation_history` も同じく `name = ForeignKey(member, ..., on_delete=SET_NULL)` です。人員削除をしても、問い合わせ・操作履歴・工数記録は「氏名だけ空欄」になって残ります。「人を消したのにデータが残っている！バグだ！」と早合点しないでください。**仕様通り** です。

6.【最重要】288枠・5分刻み設計の正体

ここが本アプリ最大の難所であり、最も賢い設計です。じっくり行きます。

6.1 なぜ288なのか

1日は24時間。24時間を **5分** で割ると…

コピー

```
24時間 × 60分 ÷ 5分 = 1440分 ÷ 5分 = 288枠
```

つまり **288 = 1日を5分刻みで区切った枠の数** です。`time_work` は `max_length=288` の文字列で、**先頭から順に 00:00-00:05, 00:05-00:10, … 23:55-24:00 の288コマ** を表します。各コマには「その5分間に何の作業をしていたか」を表す **1文字** が入ります。

コピー

```
インデックス:   0      1      2     ...    287
時間帯:      00:00  00:05  00:10  ...  23:55
           00:05  00:10  00:15  ...  24:00
time_work:   "A     A      B     ...   #"
             └作業A┘ └作業B┘        └空き┘
```

> **たとえ話:** 1日を「288個の小さなマス目が一列に並んだ細長いシート」だと思ってください。各マスは5分間。あなたは1日の終わりに「このマスは作業A、このマスは作業B、ここは休憩…」と1マスずつ塗っていきます。その塗った結果を、文字を288個並べた1本の文字列として保存しているのが `time_work` です。

6.2 各文字の意味

文字	意味
`A` 〜 `Z`, `a` 〜 `x`	工数区分（作業の種類）の記号。`kosu_division` の何番目かに対応（最大50種類）
`#`	空き（作業なし・未入力）
`$`	休憩

> **なぜ `A` 〜 `x` で50種類なのか:** `def_choice` の `def_list`（§17）や `kosu_division_dictionary`（後述）で `"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwx"` という文字列を使っています。大文字26＋小文字24（a〜x）＝50文字。これが工数区分の最大50種類（§16）に対応します。

6.3 インデックス計算の鉄則（公式）

ある時刻「HH時MM分」が288枠の **何番目（インデックス）** に当たるかは、次の公式で出します。これは `kosu_utils.py` の至る所で使われる **最重要公式** です。

コピー

```
インデックス = HH × 12 + MM ÷ 5
```

- なぜ `× 12` … 1時間 = 60分 = 12枠（5分×12）だから。1時間進むごとにインデックスは12増える。
- なぜ `MM ÷ 5` … 5分で1枠進むから。30分なら 30÷5 = 6枠ぶん。

> **▼ こう計算できれば成功:** 例「12:30」は何番目か？
>
> コピー
>
> ```
> 12 × 12 + 30 ÷ 5 = 144 + 6 = 150
> → time_work の150番目（0始まり）が12:30–12:35の枠
> ```

逆に、インデックス `t` から時刻に戻すには（`create_kosu_basic` で使用）：

コピー

```
時 = t // 12          （12で割った商）
分 = (t % 12) × 5     （12で割った余り × 5）
```

> **用語: `//`（フロア除算）と `%`（剰余）** - `//` … 割り算の **商（あまりを切り捨て）**。`150 // 12 = 12`。- `%` … 割り算の **あまり**。`150 % 12 = 6`、`6 × 5 = 30分`。よって150番目は12:30。

§8〜§9で、この公式が実コードに現れる瞬間を見ます。

7.【最重要】休憩文字列 #HHMMHHMM の読み方

`member.break_time1` や `Business_Time_graph.breaktime` に入っている **9文字の暗号** を解読します。

7.1 9文字の分解

`max_length=9` の正体は次の通りです。例として `#12451330` を分解します。

コピー

```
 #   1 2   4 5   1 3   3 0
 |   └─┘ └─┘ └─┘ └─┘
 |    |    |    |    |
接頭   開始時 開始分 終了時 終了分
(#)   (12) (45) (13) (30)

意味: 「12時45分から13時30分まで休憩」
```

位置	文字	意味
0文字目	`#`	接頭の印（休憩データの目印）
1〜2文字目	`12`	開始 **時**（HH）
3〜4文字目	`45`	開始 **分**（MM）
5〜6文字目	`13`	終了 **時**（HH）
7〜8文字目	`30`	終了 **分**（MM）

> **なぜ `#` を先頭に付けるのか:** これは「ここから休憩データですよ」という目印（センチネル）です。`parse_break_time`（§7.3）で組み立てるときも `'#' + ...` と先頭に必ず付けています。読むときは1文字目を飛ばして2文字目（インデックス1）から解釈します。

7.2「日またぎ」の罠

夜勤（3直）では、休憩が **0時をまたぐ** ことがあります。たとえば `#23450015`（23:45開始→00:15終了）。このとき、開始インデックス（23:45→285）の方が終了インデックス（00:15→3）より **大きく** なってしまいます。これを検知するのが

§9 の `break_time_process` の役目です。

7.3 文字列を作る側のコード（parse_break_time）

`kosu_utils.py` には、フロントから来た日時から、この `#HHMMHHMM` を **組み立てる** 関数があります。組み立て側を見ると、読み方がさらに腑に落ちます。

> **▼ このコードがやること（先に日本語で）**：ブラウザから送られてきた「開始日時」「終了日時」を日本時間に直し、`#開始時開始分終了時終了分` の9文字を作り、ついでに288枠でのインデックスも計算して返します。

コピー

```
def parse_break_time(break_time_start, break_time_end, jst):
  # 引数チェック: Noneや空文字列の場合
  # 開始か終了が空なら
  if not break_time_start or not break_time_end:
    # エラーを投げる
    raise ValueError("未入力箇所があります")

  try:
    # 文字列→日時
    start_time = datetime.datetime.strptime(break_time_start, "%Y-%m-%dT%H:%M:%S.%fZ")
    # 文字列→日時
    end_time = datetime.datetime.strptime(break_time_end, "%Y-%m-%dT%H:%M:%S.%fZ")
    start_time = (
        # UTCとして解釈し
        start_time.replace(tzinfo=datetime.timezone.utc)
        # 日本時間(JST)へ変換
        .astimezone(jst)
        # "HH:MM" 文字列に
        .strftime("%H:%M")
    )
    end_time = (
        # 同上（終了）
        end_time.replace(tzinfo=datetime.timezone.utc)
        .astimezone(jst)
        .strftime("%H:%M")
    )
    # ★#HHMMHHMM を組立
    time_str = '#' + start_time[0:2] + start_time[3:5] + end_time[0:2] + end_time[3:5]
    # "HH:MM"→時,分（§8）
    start_time_hour, start_time_min = time_index(start_time)
    end_time_hour, end_time_min = time_index(end_time)
    # ★開始インデックス（§6.3公式）
    start_time_ind = int(int(start_time_hour) * 12 + int(start_time_min) / 5)
    # ★終了インデックス
    end_time_ind = int(int(end_time_hour) * 12 + int(end_time_min) / 5)
  except ValueError:
    # 形式エラーは握って投げ直す
    raise ValueError("入力値の形式が不正です")

  # 開始IND, 終了IND, #HHMMHHMM を返す
  return start_time_ind, end_time_ind, time_str
```

注目すべき行： - `time_str = '#' + start_time[0:2] + start_time[3:5] + ...` `start_time` は `"12:45"` のような文字列。`[0:2]`（先頭2文字＝`"12"`）＋`[3:5]`（4〜5文字目＝`"45"`、`:` を飛ばす）で時と分を抜き、終了側も同様に並べ、頭に `#` を付ける。**これがまさに §7.1 の組み立て** です。- `start_time_ind = int(int(...) * 12 + int(...) / 5)` §6.3 の公式そのもの。「HH×12 + MM÷5」で288枠のインデックスにしています。`int(...)` で小数を切り捨て整数化。

> **用語: `raise` （レイズ：例外を投げる）** … 「これは異常事態だ」とプログラムにエラーを通知すること。呼び出した側が `try / except` で受け止めて、画面にエラーメッセージを出します。

これで「休憩は9文字の暗号で保存され、読むときは2文字目から時・分を取り、288枠インデックスに変換する」という全体像がつかめました。次節から、その変換を担う関数を1つずつ精読します。

---

8. kosu_utils.py 逐行解説①: time_index

最も基本の小さな関数から。

> **▼ このコードがやること（先に日本語で）:** `"12:45"` のような「時:分」の文字列を受け取り、`:` の位置を境に「時の部分」と「分の部分」に切り分けて返します。

コピー

```
# 時＆分 分離関数
# 引数: "HH:MM" の文字列
def time_index(post_time):
  # 時間の区切りのインデックス取得
  # ':' が何文字目かを探す（"12:45"なら2）
  post_index = post_time.index(':')
  # 時取得
  # 先頭から ':' の手前まで → "12"
  time_hour = post_time[ : post_index]
  # 分取得
  # ':' の次から末尾まで → "45"
  time_min = post_time[post_index + 1 : ]

  # 時と分返す
  # ("12", "45") を返す
  return time_hour, time_min
```

1行ずつ： - `post_index = post_time.index(':')` … 文字列メソッド `.index(':')` は「`:` が最初に現れる位置（0始まり）」を返す。`"12:45"` なら `2`。 - `time_hour = post_time[ : post_index]` … **スライス**。「先頭から `post_index`（=2）の手前まで」＝ `"12"`。 - `time_min = post_time[post_index + 1 : ]` … 「`post_index+1`（=3）から末尾まで」＝ `"45"`（`:` を1個飛ばす）。 - `return time_hour, time_min` … 2つの値をまとめて返す（タプル）。

用語: スライス（slice） … 文字列やリストの一部を切り出す記法。`文字列[開始:終了]` で、開始位置から「終了の1つ手前」までを取り出します。`"12:45"[0:2]` は `"12"`。終了を省くと末尾まで、開始を省くと先頭から。

▼ こう動けば成功:

コピー

```
time_index("09:30")  →  ("09", "30")
time_index("23:05")  →  ("23", "05")
```

なぜわざわざ関数に？ たった3行ですが、`parse_break_time`（§7.3）でも何度も「時:分を分ける」操作が出てきます。**同じ処理を1か所にまとめて名前を付ける**（部品化）ことで、間違いが減り、読みやすくなります。

---

9. kosu_utils.py 逐行解説②: break_time_process

休憩文字列を288枠インデックスに変換し、**日またぎ** を判定する関数です。§7.2 の罠を解決します。

▼ このコードがやること（先に日本語で）: `#HHMMHHMM` の休憩文字列から「休憩開始の枠インデックス」「休憩終了の枠インデックス」を計算し、開始の方が終了より遅ければ「0時をまたいでいる（日またぎ）」と判定して旗を立てます。

コピー

```
# 休憩時間のインデックス日またぎチェック関数
# 引数: "#HHMMHHMM"
def break_time_process(breaktime_str):
  # 休憩開始時間のインデックス取得
  # 開始HH×12 + 開始MM÷5
  break_start = int(breaktime_str[1 : 3])*12 + int(breaktime_str[3 : 5])/5
  # 休憩終了時間のインデックス取得
  # 終了HH×12 + 終了MM÷5
  break_end = int(breaktime_str[5 : 7])*12 + int(breaktime_str[7 :])/5

  # 休憩の日またぎ変数リセット
  # 日またぎ旗を0で初期化
  break_next_day = 0
  # 休憩開始時間より終了時間の方が早い場合の処理
  # 開始 > 終了 なら（例:23:45→00:15）
  if break_start > break_end:
    # 日またぎ変数に1を入れる
    # 日をまたいだ印を立てる
    break_next_day = 1

  # 休憩開始時間より終了時間の方が遅い場合の処理
  # そうでなければ（普通の昼休み）
  else:
    # 日またぎ変数に0を入れる
    # またいでいない
    break_next_day = 0

  # 開始IND, 終了IND, 日またぎ旗
  return break_start, break_end, break_next_day
```

ここがこの章の核心の1つ。**文字列のスライス位置** に注目してください。

- `breaktime_str[1 : 3]` … インデックス1〜2文字目 ＝ **開始の時**（HH）。0文字目の `#` を **飛ばしている** のがポイント。
- `breaktime_str[3 : 5]` … 3〜4文字目 ＝ **開始の分**（MM）。
- `breaktime_str[5 : 7]` … 5〜6文字目 ＝ **終了の時**（HH）。
- `breaktime_str[7 : ]` … 7文字目から末尾 ＝ **終了の分**（MM）。

これはまさに §7.1 の表の通りの分解です。そして各行は §6.3 の公式 `HH×12 + MM÷5` で288枠インデックスに変換しています。

▼ **こう動けば成功**: 入力 `"#12451330"` （12:45-13:30）の場合：

コピー

```
break_start = 12×12 + 45/5 = 144 + 9  = 153.0
break_end   = 13×12 + 30/5 = 156 + 6  = 162.0
153 < 162 なので break_next_day = 0（日またぎなし）
```

入力 `"#23450015"` （23:45-00:15）の場合：

コピー

```
break_start = 23×12 + 45/5 = 276 + 9  = 285.0
break_end   =  0×12 + 15/5 =   0 + 3  =   3.0
285 > 3 なので break_next_day = 1（日またぎ！）
```

⚠️ **なぜ日またぎを判定する必要が？**「285番目から3番目まで休憩」と素直にループすると、285→3は逆走で1回も回りません。日またぎ旗が立っていれば、呼び出し側で「285→287（その日の終わり）」と「0→3（翌日の頭）」の2回に分けて処理できます。夜勤の休憩を正しく扱うための必須の仕掛けです。

---

10. kosu_utils.py 逐行解説③: kosu_write

288枠の「作業内容リスト」と「作業詳細リスト」に、入力された工数を **書き込む** 関数です。

▼ **このコードがやること（先に日本語で）**:「開始枠から終了枠まで」の各マスに、入力された作業記号と作業詳細を順番に塗っていきます。§6.1の「マスを塗る」操作そのものです。

コピー

```
# 工数書き込み関数
# 開始枠,終了枠,作業内容List,詳細List,入力
def kosu_write(start_ind, end_ind, kosu_def, detail_list, post_data):
  # 作業内容と作業詳細を書き込むループ
  # 開始枠から終了枠の手前までループ
  for kosu in range(start_ind, end_ind):
    # 作業内容リストに入力された工数定義区分の対応する記号を入れる
    # そのマスに作業記号を塗る
    kosu_def[kosu] = post_data.get('time_work')
    # 作業詳細リストに入力した作業詳細を入れる
    # そのマスに作業詳細を塗る
    detail_list[kosu] = post_data.get('detail_work')

  # 塗り終えた2つのリストを返す
  return kosu_def, detail_list
```

- `for kosu in range(start_ind, end_ind):` … `range(開始, 終了)` は「開始から **終了の1つ手前まで**」の連番を生成。たとえば `range(150, 153)` は `150, 151, 152`（153は含まない）。**だから終了枠ちょうどは塗られない**（終了時刻の5分前まで）。
- `kosu_def[kosu] = post_data.get('time_work')` … リストの `kosu` 番目の要素を、入力された作業記号（`A` など1文字）に書き換え。
- `detail_list[kosu] = post_data.get('detail_work')` … 詳細リストの同じ位置に作業詳細の文字列を入れる。
- `post_data.get('time_work')` … `post_data`（フロントから来た入力データの辞書）から `'time_work'` の値を取り出す。`.get()` はキーが無くてもエラーにならず `None` を返す安全な取り出し方。

> **用語: リスト（list）** … 複数の値を順番に並べて持つ入れ物。`kosu_def[3]` のように **番号（インデックス）で各要素を読み書き** できます。`time_work` 文字列を `list(...)` で1文字ずつのリストにほどいて使います（§14の `kosu_sort` で `list(obj_get.time_work)` しているのがそれ）。
>
> **▼ こう動けば成功:** `kosu_write(150, 153, kosu_def, detail_list, {'time_work':'A','detail_work':'部品交換'})` を呼ぶと、`kosu_def[150], kosu_def[151], kosu_def[152]` が `'A'` に、対応する詳細が `'部品交換'` になります（12:30〜12:45の15分が作業A）。

---

11. kosu_utils.py 逐行解説④: break_time_delete

休憩枠に工数が入っていたときの扱いを決める関数。§4.6 の `break_check` がここで効いてきます。

> **▼ このコードがやること（先に日本語で）:** 休憩の枠を1マスずつ見て、（A）休憩エラー有効の人なら「休憩中に作業が入っていたらエラーを返す」、（B）無効の人なら「休憩枠の作業を黙って消す」という処理をします。

コピー

```
# 休憩範囲工数削除関数
# 開始,終了,作業List,詳細List,人員
def break_time_delete(break_start_ind, break_end_ind, work_list, detail_list, member_obj):
  # 休憩時間ループ
  # 休憩開始枠～終了枠の手前まで
  for bt in range(int(break_start_ind), int(break_end_ind)):
    # ユーザーが休憩エラー有効チェックONの場合の処理
    # この人が「休憩エラー有効」なら
    if member_obj.break_check == True:
      # 作業内容リストが空でない場合の処理
      # そのマスが空(#)でない かつ
      if work_list[bt] != '#':
        # 作業内容リストが休憩でない場合の処理
        # 休憩($)でもない（=作業が入っている）なら
        if work_list[bt] != '$':
          # エラーを返す
          return '休憩時間に工数を入力できません。休憩変更するor休憩エラー無効で入力できます。', None, None

    # ユーザーが休憩エラー有効チェックOFFの場合の処理
    # 「休憩エラー無効」なら
    else:
      # 作業内容リストの要素を空にする
      # そのマスを空(#)に塗りつぶす
      work_list[bt] = '#'
      # 作業詳細リストの要素を空にする
      # 詳細も空に
      detail_list[bt] = ''

  # エラーなし，更新後の2リスト
  return None, work_list, detail_list
```

1行ずつ： - `for bt in range(int(break_start_ind), int(break_end_ind)):` … 休憩の各マスをループ。`int(...)` は §9 で計算したインデックスが小数（`153.0`）なので整数に直すため。- `if member_obj.break_check == True:` … この人（`member_obj`）の `break_check` フラグ（§4.6）で分岐。- 有効（True）の枝：`work_list[bt] != '#'`（空でない）かつ `!= '$'`（休憩でない）＝**作業が入っている** なら、`return 'エラー文', None, None` で **即座に処理を打ち切ってエラーを返す**。- 無効（else）の枝：何も言わず `work_list[bt] = '#'`（空に）、`detail_list[bt] = ''`（詳細も空に）。つまり休憩枠の作業を **上書き消去** する。- `return None, work_list, detail_list` … 最後まで問題なければ、エラーなし（`None`）と更新済みの2リストを返す。

> **戻り値が3つある理由:** 呼び出し側は「1つ目がエラー文字列か `None` か」を見て、エラーなら画面に表示、`None` なら2・3番目の更新済みリストを使って保存処理を続けます。「結果＋データ」を1度に返すPythonらしい書き方です。
>
> ⚠️ **`break_check` の業務的意味:** 「休憩中は絶対に工数を入れさせたくない」厳格な部署は `break_check=True`。「実態に合わせて休憩中も作業したことにしたい／自動でクリアしてほしい」部署は `False`。**1つのフラグで運用ポリシーを切り替える** 良い例です。後任者がこの分岐を消すと、片方の運用が成立しなくなります。

12. kosu_utils.py 逐行解説⑤: break_time_write

休憩の枠を `$`（休憩）で塗り直す関数です。`break_time_delete` が「消す」のに対し、こちらは「休憩で埋める」。

> **▼ このコードがやること（先に日本語で）**: 休憩の開始枠から終了枠まで、各マスを `$`（＝休憩）に書き換え、作業詳細を空にします。休憩を確定させる処理です。

コピー

```
# 休憩書き込み関数
# 開始枠,終了枠,作業List,詳細List
def break_time_write(break_start_ind, break_end_ind, kosu_def, detail_list):
  # 休憩時間内の工数データを休憩に書き換えるループ
  # 休憩の各マス
  for bt in range(int(break_start_ind), int(break_end_ind)):
    # 作業内容リストの要素を休憩に書き換え
    # そのマスを「休憩($)」にする
    kosu_def[bt] = '$'
    # 詳細は空に
    detail_list[bt] = ''

  # 更新後の2リストを返す
  return kosu_def, detail_list
```

- `kosu_def[bt] = '$'` … 休憩の各マスを `$`（§6.2の休憩記号）で塗る。
- `detail_list[bt] = ''` … 休憩マスに作業詳細は不要なので空文字に。

> **`break_time_delete` との違い（重要）**: - `break_time_delete` … 休憩枠を `'#'`（空き）にする or エラーを返す。「作業を取り除く」役。- `break_time_write` … 休憩枠を `'$'`（休憩）にする。「ここは休憩だと明示する」役。工数合計の計算（§14）では `#`（空き）と `$`（休憩）の **両方** を勤務時間から差し引くので、見た目の集計は似ますが、`$` は「意図的な休憩」、`#` は「未入力・空き」と意味が違います。混同しないこと。
>
> **なぜ休憩を `$` で明示するのか**: §14 の `judgement_check` で `kosu_def.count('$')*5` として **休憩分の分数** を引き算します。`$` の個数 × 5分 が休憩時間。だから休憩はきちんと `$` で塗っておく必要があります。

---

13. kosu_utils.py 逐行解説⑥: detail_list_summarize

288個に分かれた「作業詳細リスト」を、**1本の文字列に連結** してデータベースに保存できる形にする関数です。`detail_work` カラム（§5）の中身を作ります。

▼ **このコードがやること（先に日本語で）**：288個の作業詳細を、区切り文字 `$` でつなげて1本の長い文字列にします。最後の要素のうしろには `$` を付けません。

コピー

```
# 作業詳細リスト文字列変換関数
# 引数: 288個の作業詳細リスト
def detail_list_summarize(detail_list):
  # 作業詳細str型定義
  # 連結結果を入れる空文字列
  detail_list_str = ''

  # 作業詳細リストをstr型に変更するループ
  # 各要素を番号i付きで取り出す
  for i, e in enumerate(detail_list):
    # 最終ループの処理
    # 最後の要素なら
    if i == len(detail_list) - 1:
      # 作業詳細変数に作業詳細リストの要素をstr型で追加する
      # 区切りなしで追加（末尾に$は付けない）
      detail_list_str = detail_list_str + detail_list[i]

    # 最終ループ以外の処理
    # 最後以外なら
    else:
      # 作業詳細変数に作業詳細リストの要素をstr型で追加し、区切り文字の'$'も追加
      # 要素 + 区切り'$' を追加
      detail_list_str = detail_list_str + detail_list[i] + '$'

  # つなげた1本の文字列を返す
  return detail_list_str
```

- `enumerate(detail_list)` … リストを「番号 `i` と中身 `e` のペア」で順に取り出す関数。`i` が今何番目かを教えてくれます。
- `if i == len(detail_list) - 1:` … `len(...)` はリストの長さ（288）。`288 - 1 = 287` が **最後のインデックス**。最後だけ特別扱い。
- 最後の要素 … `detail_list_str + detail_list[i]`（`$` を付けずに連結）。
- それ以外 … `detail_list_str + detail_list[i] + '$'`（`$` 区切りを付けて連結）。

なぜ末尾に `$` を付けないのか: 後で読み戻すとき（ `kosu_sort` の `obj_get.detail_work.split('$')` ）に、`A$B$C` を `$` で分割すれば `['A','B','C']` と **ちょうど288個** に戻せます。末尾に余分な `$` があると `['A','B','C','']` と289個になり、枠数がズレてしまいます。**区切り文字方式の鉄則**です。

⚠️ **`$` が二役なことに注意**: `$` は「休憩記号（ `time_work` 内）」でもあり「作業詳細の区切り（ `detail_work` 内）」でもあります。**入っているカラムが違う** ので衝突しませんが、後任者が文字を変えるときは両方の意味を意識してください。

---

14. kosu_utils.py 逐行解説⑦: judgement_check（工数整合性判定）

この章で最も長く、最も業務ルールが詰まった関数。「**入力された工数が、勤務形態・直・残業時間と矛盾していないか**」を判定し、`Business_Time_graph.judgement`（OK/NG）に入れる値を決めます。

14.1 工数合計の計算（冒頭）

▼ このコードがやること（先に日本語で）: まず288枠のうち「空き(#)」と「休憩($)」を除いた残り＝実作業の合計を分で出します。そこから勤務形態ごとに「あるべき分数」と一致するかを次々チェックし、合えば `judgement=True`（OK）にします。

コピー

```
# 工数データ整合性判断関数
# 作業List,就業形態,直,人員,残業
def judgement_check(kosu_def, work, tyoku, member_obj, over_work):
  # 工数合計取得
  # 1日1440分 - 空き分 - 休憩分
  kosu_total = 1440 - (kosu_def.count('#')*5) - (kosu_def.count('$')*5)

  # 工数入力OK_NGリセット
  # まずNG(False)から始める
  judgement = False
```

- `kosu_total = 1440 - (kosu_def.count('#')*5) - (kosu_def.count('$')*5)`
- `1440` … 1日の総分数（24時間×60分）。
- `kosu_def.count('#')` … 288枠のうち「空き」の個数。`×5` で空き分の分数。
- `kosu_def.count('$')` … 「休憩」の個数。`×5` で休憩分の分数。
- 1440から空きと休憩を引けば、**残り＝実際に作業した分数**。これが `kosu_total`。
- `judgement = False` … いったんNGに。以下のチェックのどれかに合致したらTrueに上げる方式。

14.2 出勤・休出・早退遅刻の判定

コピー

```
# 出勤、休出時、工数合計と残業に整合性がある場合の処理
# 就業形態が出勤系で
if (work == '出勤' or work == 'シフト出') and \
  # 実作業 - 残業 == 470分(定時) なら
  kosu_total - int(over_work) == 470:
  # 工数入力OK_NGをOKに切り替え
  # OK
  judgement = True

# 休出時、工数合計と残業に整合性がある場合の処理
# 休出は全部残業扱い
if work == '休出' and kosu_total == int(over_work) and int(over_work) != 0:
  # OK
  judgement = True

# 早退・遅刻時、工数合計と残業に整合性がある場合の処理
# 早退遅刻は作業が少しでもあればOK
if work == '早退・遅刻' and kosu_total != 0:
  judgement = True
```

- `(work == '出勤' or work == 'シフト出')` … 就業形態が「出勤」か「シフト出」。
- `kosu_total - int(over_work) == 470` … **実作業時間から残業を引いた値が470分（=7時間50分）＝定時の所定労働時間** なら整合。`int(over_work)` は残業を整数化。
- `\`（行末のバックスラッシュ）… 「この行はまだ続く」という継続記号。長い `if` を改行して読みやすくしているだけ。
- 休出 … `kosu_total == int(over_work)` かつ残業が0でない＝**働いた時間が全部残業として登録されている**ならOK。
- 早退・遅刻 … 少しでも作業があれば（`kosu_total != 0`）OK。

> **なぜ470分なのか:** この会社の定時（昼休みを除いた所定労働）が470分（7時間50分）に設定されているためです。この数字は **業務の取り決め** であり、勝手に変えると全社員の工数判定が狂います。

14.3 常昼・連2 の半年休判定

コピー

```
# 常昼の場合の処理
# 直が常昼(4)
if tyoku == '4':
  # 半前年休時、工数合計と残業に整合性がある場合の処理
  # 午前半休→残り230分
  if work == '半前年休' and kosu_total - int(over_work) == 230:
    judgement = True
  # 半後年休時、工数合計と残業に整合性がある場合の処理
  # 午後半休→残り240分
  if work == '半後年休' and kosu_total - int(over_work) == 240:
    judgement = True

# 連2の場合の処理
# 直が連1(5)/連2(6)
if tyoku == '5' or tyoku == '6':
  # 連操の午前半休→220分
  if work == '半前年休' and kosu_total - int(over_work) == 220:
    judgement = True
  # 連操の午後半休→250分
  if work == '半後年休' and kosu_total - int(over_work) == 250:
    judgement = True
```

- `tyoku == '4'` … 直が常昼。半休（半年休）のとき、午前休なら残り作業が230分、午後休なら240分が整合。
- 半休の分数が「午前(230)」「午後(240)」で違うのは、午前と午後で実働時間が違うから。
- `tyoku == '5' or '6'`（連操）では基準が220/250と変わる。**勤務形態ごとに所定時間が違う** ので、判定値も全部別々に書いてあります。

14.4 ショップ × 直 ごとの半年休判定

ここから先は **ショップ（§4.1）と直の組み合わせ** で、半休の基準分数を細かく分けています。長いですが構造は同じ繰り返しです。代表として最初のブロックを精読します。

なぞってコピーすると「ＩＩ」が別の文字になります。正しくは → コピー

```
# ログイン者の登録ショップが三組三交替ＩＩ甲乙丙番Cで1直の場合の処理
# ショップがW系/A系/J/組長(W,A)で
if (member_obj.shop == 'W1' or member_obj.shop == 'W2' or \
  member_obj.shop == 'A1' or member_obj.shop == 'A2' or \
    member_obj.shop == 'J' or member_obj.shop == '組長以上(W,A)') and \
    # かつ 1直 なら
    tyoku == '1':
  # 午前半休 → 230分
  if work == '半前年休' and kosu_total - int(over_work) == 230:
    judgement = True
  # 午後半休 → 240分
  if work == '半後年休' and kosu_total - int(over_work) == 240:
    judgement = True
```

- ショップ判定の `( ... or ... or ... )` … `member_obj.shop` が C番系（W1,W2,A1,A2,J,組長(W,A)）のどれかに一致するか。
- `and tyoku == '1'` … さらに1直のとき。
- 半前年休→230分、半後年休→240分が整合の基準。

以降、同じ形で次の組み合わせが並びます（基準値だけが変わる）。一覧にすると理解しやすいです。

ショップ群	直	半前年休の基準	半後年休の基準
C番系(W1,W2,A1,A2,J,組長(W,A))	1直	230	240
C番系	2直	290	180
C番系	3直	230	240
B番系(P,R,T1,T2,その他,組長(P,R,T,その他))	1直	220	250
B番系	2直	230	240
B番系	3直	275	195

> ⚠️ **これは「魔法の数字（マジックナンバー）」の塊です。** 220, 230, 240, 250, 275, 290, 180, 195, 470… これらはすべて **会社の勤務シフトと所定労働時間から導かれた実数値** です。シフトの取り決めが変わったときだけ、対応する数字を変更します。**コードを読んで意味が分からない数字こそ、その部署の勤務ルールそのもの** だと理解してください。

14.5 休み系の判定（末尾）

コピー

```
# 出勤、休出時、工数合計と残業に整合性がある場合の処理
if (work == '休日' or work == 'シフト休' or work == '年休' or work == '代休' or work == '公休' or work == '欠勤') and \
  # 休み系は実作業が0分ならOK
  kosu_total == 0:
  judgement = True

# 最終的なOK/NGを返す
return judgement
```

- 休日・シフト休・年休・代休・公休・欠勤 … いずれも「働いていない日」。`kosu_total == 0`（実作業ゼロ）なら整合でOK。
- `return judgement` … ここまでのチェックを1つでも通っていれば `True`、どれも通らなければ `False` を返す。これが `Business_Time_graph.judgement` に保存されます。

> ▼ **こう動けば成功:** 1直で普通に出勤、残業0分、休憩45分、実作業7時間50分(470分)を入力すると…
>
> コピー
>
> ```
> kosu_total = 1440 - (空き分) - (休憩45分) = ... = 470
> work=='出勤' かつ 470 - 0 == 470 → judgement = True（OK）
> ```
>
> 工数が1枠足りない（465分）なら、どの条件にも一致せず `judgement = False`（NG）。画面に「工数が合いません」と出る、という仕組みです。
>
> **保守のキモ:** バグ報告「正しく入力したのにNGになる」が来たら、まず `judgement_check` の **そのショップ×直×就業形態の基準値** と、実際の `kosu_total`（休憩・空きの数え方）を疑います。直近コミット `休憩変更時の工数消去バグ修正` (9156f15) も、休憩の塗り方が `kosu_total` を狂わせていた類の問題です。

---

15. モデル③: team_member（チーム）

組長などが「自分のチームメンバー（最大15人）」を登録する表です。

> ▼ **このコードがやること（先に日本語で）:** チームを組む人（`employee_no5`）に対し、メンバーの従業員番号を最大15人ぶん登録します。フォロー機能のON/OFFも持ちます。

コピー

```
# 「チーム構成」の表
class team_member(models.Model):
  # チームの持ち主（組長など）の番号
  employee_no5 = models.IntegerField('従業員番号')
  # メンバー1
  member1 = models.CharField('メンバー従業員番号1', max_length=6, blank=True, null=True)
  # メンバー2
  member2 = models.CharField('メンバー従業員番号2', max_length=6, blank=True, null=True)
  # メンバー3
  member3 = models.CharField('メンバー従業員番号3', max_length=6, blank=True, null=True)
  # メンバー4
  member4 = models.CharField('メンバー従業員番号4', max_length=6, blank=True, null=True)
  # メンバー5
  member5 = models.CharField('メンバー従業員番号5', max_length=6, blank=True, null=True)
  # メンバー6
  member6 = models.CharField('メンバー従業員番号6', max_length=6, blank=True, null=True)
  # メンバー7
  member7 = models.CharField('メンバー従業員番号7', max_length=6, blank=True, null=True)
  # メンバー8
  member8 = models.CharField('メンバー従業員番号8', max_length=6, blank=True, null=True)
  # メンバー9
  member9 = models.CharField('メンバー従業員番号9', max_length=6, blank=True, null=True)
  # メンバー10
  member10 = models.CharField('メンバー従業員番号10', max_length=6, blank=True, null=True)
  # メンバー11
  member11 = models.CharField('メンバー従業員番号11', max_length=6, blank=True, null=True)
  # メンバー12
  member12 = models.CharField('メンバー従業員番号12', max_length=6, blank=True, null=True)
  # メンバー13
  member13 = models.CharField('メンバー従業員番号13', max_length=6, blank=True, null=True)
  # メンバー14
  member14 = models.CharField('メンバー従業員番号14', max_length=6, blank=True, null=True)
  # メンバー15
  member15 = models.CharField('メンバー従業員番号15', max_length=6, blank=True, null=True)
  # フォロー機能の有効/無効
  follow = models.BooleanField('フォローON/OFF', null=True)

  # 文字表現
  def __str__(self):
    # 持ち主の従業員番号を返す
    return str(self.employee_no5)
```

- `employee_no5` … このチームの持ち主（組長など）の従業員番号。
- `member1` ～ `member15` … メンバーの従業員番号を **文字列** で（最大6文字）。空き枠は `blank=True, null=True` で未登録OK。
- `follow` … フォロー機能のスイッチ。チームメンバーの工数入力状況を追う機能のON/OFF。

> **なぜメンバーを番号の文字列で持つのか（ForeignKeyにしない理由）**：15個の `ForeignKey` を作る代わりに、番号を文字列で持つ簡易な設計です。利点は単純さ。欠点は「番号から人員を引くときに自分でコードを書く必要がある」点。本アプリは規模が小さく15枠固定で足りるので、この割り切りが採用されています。
>
> ⚠️ **`max_length=6`**：従業員番号は最大6桁の想定。番号体系が7桁以上に変わると、ここを広げるマイグレーションが必要です。

---

16. モデル④: kosu_division（工数区分定義・最大50区分）

「作業の種類（工数区分）」を **最大50個** 定義する表です。`time_work` の `A` 〜 `x` の各文字が、ここの何番目の区分かを指します。

16.1 構造（1区分 = 4フィールドのセット）

> **▼ このコードがやること（先に日本語で）**：1つのバージョン名（Ver）の下に、「区分名・定義・作業内容・変動/固定」の4点セットを最大50区分ぶん持ちます。冒頭の1〜2区分と末尾の50区分目だけ実コードを示し、3〜49区分は同じ形の繰り返しです。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
# 「工数区分定義」の表
class kosu_division(models.Model):
  # ★バージョン名(Ver)
  kosu_name = models.CharField('工数区分定義Ver名', blank=True, null=True, max_length=100)

  # 1区分目の名前
  kosu_title_1 = models.CharField('工数区分名1', blank=True, null=True, max_length=100)
  # 1区分目の定義(説明)
  kosu_division_1_1 = models.TextField('定義1', blank=True, null=True)
  # 1区分目の作業内容
  kosu_division_2_1 = models.TextField('作業内容1', blank=True, null=True)
  # 1区分目が変動か固定か
  kosu_division_3_1 = models.BooleanField('変動/固定1', null=True)

  # 2区分目の名前
  kosu_title_2 = models.CharField('工数区分名2', blank=True, null=True, max_length=100)
  # 2区分目の定義
  kosu_division_1_2 = models.TextField('定義2', blank=True, null=True)
  # 2区分目の作業内容
  kosu_division_2_2 = models.TextField('作業内容2', blank=True, null=True)
  # 2区分目が変動/固定
  kosu_division_3_2 = models.BooleanField('変動/固定2', null=True)
  # …（3区分目～49区分目まで、まったく同じ4フィールドのセットが繰り返される）…
  # 50区分目の名前
  kosu_title_50 = models.CharField('工数区分名50', blank=True, null=True, max_length=100)
  # 50区分目の定義
  kosu_division_1_50 = models.TextField('定義50', blank=True, null=True)
  # 50区分目の作業内容
  kosu_division_2_50 = models.TextField('作業内容50', blank=True, null=True)
  # 50区分目が変動/固定
  kosu_division_3_50 = models.BooleanField('変動/固定50', null=True)

  # 文字表現
  def __str__(self):
    # 「id : Ver名」
    return str(self.id) + ' : ' + str(self.kosu_name)
```

各区分の4フィールドの命名規則：

フィールド名	意味
`kosu_title_N`	N番目の区分名（例「段取り」「検査」）
`kosu_division_1_N`	N番目の「定義」（この区分は何を指すかの説明）
`kosu_division_2_N`	N番目の「作業内容」（具体的にどんな作業か）
`kosu_division_3_N`	N番目の「変動/固定」（True/Falseのフラグ）

> **N と記号の対応:** `kosu_title_1` ↔ 記号 `A`、`kosu_title_2` ↔ `B`、…、`kosu_title_26` ↔ `Z`、`kosu_title_27` ↔ `a`、…、`kosu_title_50` ↔ `x`。`time_work` 内の1文字を、この対応表で「人間が読める区分名」に翻訳します（次の `kosu_division_dictionary` がそれ）。

16.2 Ver（バージョン）管理という考え方

`kosu_name` が **バージョン名**。1レコード = 1バージョンの定義一式です。

> **なぜバージョン管理が必要か（最重要）:** 工数区分は **年度や工程変更で中身が変わります**。たとえば2025年は「区分A=段取り」だったのが、2026年に「区分A=保全」に変わったとします。もし定義を1つしか持てないと、**過去の `time_work` に入っている `A` の意味が変わってしまい、昔の集計が狂う** のです。そこで、定義を丸ごと別レコード（別Ver）として保存し、各工数記録（`Business_Time_graph.def_ver2`）に **そのとき使ったVer名** を刻んでおきます。これで「2025年のAは段取り、2026年のAは保全」と正しく解釈できます。**過去を壊さずに定義を更新する仕組み** がVer管理です。

16.3 定義を読む側のコード（kosu_division_dictionary）

`time_work` の1文字を区分名に翻訳し、登録されている区分数を数える関数です。

> **▼ このコードがやること（先に日本語で）:** 指定されたVer名の定義を取り出し、「何区分まで登録されているか」を数え、`[記号, 区分名]` のペアのリストを作って返します。

コピー

```
# 工数区分定義辞書作成関数
def kosu_division_dictionary(def_name):
    # 現在使用している工数区分のオブジェクトを取得
    # Ver名で定義1件を取得
    kosu_obj = kosu_division.objects.get(kosu_name=def_name)

    # 工数区分登録カウンターリセット
    # 登録区分数カウンタ
    n = 0
    # 工数区分登録数カウント
    # 1〜50区分をチェック
    for kosu_num in range(1, 51):
      # kosu_title_N の値を動的に取得
      val = getattr(kosu_obj, f'kosu_title_{kosu_num}')
      # 名前が入っていれば
      if val not in [None, '']:
        # そこまで登録ありと記録
        n = kosu_num

    # 工数区分処理用記号リスト用意
    # 記号を登録数ぶん
    str_list = list("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwx")[:n]

    # 工数区分の選択リスト作成
    # [記号,区分名]のリスト
    choices_list = []
    # 各記号について
    for i, m in enumerate(str_list):
      # 対応する区分名を取得
      title_val = getattr(kosu_obj, f'kosu_title_{i + 1}')
      # [記号, 区分名] を追加
      choices_list.append([m, title_val])

    # ペアのリストと区分数を返す
    return choices_list, n
```

注目： - `kosu_division.objects.get(kosu_name=def_name)` … **ORM（オーアールエム：Object-Relational Mapping）** を使い、Ver名で1件取得。SQLを書かずにPythonで「WHERE kosu_name = def_name」を実現しています。- `getattr(kosu_obj, f'kosu_title_{kosu_num}')` … `getattr(オブジェクト, "属性名の文字列")` は、**フィールド名を文字列で組み立てて値を取る** 関数。`f'kosu_title_{kosu_num}'` で `'kosu_title_1'` 〜 `'kosu_title_50'` を作り、50個のフィールドをループで回せます。**「番号付きフィールドが大量にある」設計だからこそ生きるテクニック**です。- `str_list = list("ABC...x")[:n]` … 記号50文字のうち、登録されている区分数 `n` 個ぶんだけ切り出す。

> **用語: ORM（Object-Relational Mapping）** … Pythonのオブジェクト操作を、自動でデータベースのSQLに翻訳する仕組み。`.objects.get(...)` `.objects.filter(...)` `.objects.count()` などがその窓口です。

---

17. モデル⑤: def_choice（作業詳細の選択肢）

工数の「作業詳細」を選ぶときの、記号と説明のマスタ（基準データ）です。

> **▼ このコードがやること（先に日本語で）:** 「定義記号（A〜x と $ の50種）」と、その記号が表す作業詳細の説明文を1組として登録します。

コピー

```
# 「作業詳細の選択肢」の表
class def_choice(models.Model):
  # 記号の選択肢
  def_list = [(x, x) for x in "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwx$"]

  # 記号（A〜x,$）
  def_symbol = models.CharField('定義記号', choices=def_list, max_length=2)
  # その記号の作業詳細(説明)
  def_select = models.CharField('作業詳細', max_length=100)

  # 文字表現
  def __str__(self):
    # 「記号＋詳細」
    return str(self.def_symbol) + str(self.def_select)
```

- `def_list = [(x, x) for x in "ABC...$"]` … **リスト内包表記**。文字列 `"ABC...x$"` の各文字 `x` について `(x, x)` を作る、を一気にやる書き方。`[('A','A'), ('B','B'), ..., ('$','$')]` が生成されます。
- `def_symbol` … 記号。`choices=def_list` でプルダウン化、`max_length=2`。
- `def_select` … その記号が表す作業詳細（最大100文字）。

> **用語: リスト内包表記（list comprehension）** … `[式 for 変数 in 並び]` の形で、ループを1行で書いてリストを作る記法。`[(x, x) for x in "AB"]` は `[('A','A'),('B','B')]`。短くて速い、Python頻出の書き方です。
>
> **`time_work` の文字との関係:** `def_choice` の記号集合（`A` 〜 `x` ＋ `$`）は、`time_work` に入る文字（§6.2）と整合しています。`$`（休憩）も選択肢に含まれているのがポイントです。

---

18. モデル⑥: administrator_data（システム設定）

アプリ全体の設定（1件だけ存在する想定）を保持する表です。

> **▼ このコードがやること（先に日本語で）**：一覧の表示件数、問い合わせの担当者番号（最大3名）、全体お知らせ（最大5件）など、システム全体の設定を1か所にまとめます。

コピー

```
# 「システム設定」の表
class administrator_data(models.Model):
  # 一覧の1ページ表示件数
  menu_row = models.CharField('一覧表示項目数', max_length=4)
  # 担当者1
  administrator_employee_no1 =  models.CharField('問い合わせ担当者従業員番号1', max_length=255, null=True, blank=True)
  # 担当者2
  administrator_employee_no2 =  models.CharField('問い合わせ担当者従業員番号2', max_length=255, null=True, blank=True)
  # 担当者3
  administrator_employee_no3 =  models.CharField('問い合わせ担当者従業員番号3', max_length=255, null=True, blank=True)
  # 全体お知らせ1
  pop_up1 = models.CharField('ポップアップ1', max_length=255, null=True, blank=True)
  # お知らせID1
  pop_up_id1 =  models.CharField('ポップアップID1', max_length=255, null=True, blank=True)
  # 全体お知らせ2
  pop_up2 = models.CharField('ポップアップ2', max_length=255, null=True, blank=True)
  # お知らせID2
  pop_up_id2 =  models.CharField('ポップアップID2', max_length=255, null=True, blank=True)
  # 全体お知らせ3
  pop_up3 = models.CharField('ポップアップ3', max_length=255, null=True, blank=True)
  # お知らせID3
  pop_up_id3 =  models.CharField('ポップアップID3', max_length=255, null=True, blank=True)
  # 全体お知らせ4
  pop_up4 = models.CharField('ポップアップ4', max_length=255, null=True, blank=True)
  # お知らせID4
  pop_up_id4 =  models.CharField('ポップアップID4', max_length=255, null=True, blank=True)
  # 全体お知らせ5
  pop_up5 = models.CharField('ポップアップ5', max_length=255, null=True, blank=True)
  # お知らせID5
  pop_up_id5 =  models.CharField('ポップアップID5', max_length=255, null=True, blank=True)

  # 文字表現
  def __str__(self):
    # 「設定1」など
    return '設定' + str(self.id)
```

- `menu_row` … 一覧画面の1ページあたり表示件数（最大4文字＝数千件まで）。
- `administrator_employee_no1〜3` … 問い合わせ（`inquiry_data`）を受ける担当者の従業員番号。最大3名。
- `pop_up1〜5` ＋ `pop_up_id1〜5` … 全社員向けの一斉お知らせ（`member` の個人ポップアップと別物の、システム全体版）。

> **なぜ1件だけの設定をテーブルにするのか:**「設定値もデータベースに置けば、コードを書き換えずに管理画面から変更できる」からです。表示件数や担当者を運用中に変えたいとき、再デプロイ不要で済みます。

19. モデル⑦: inquiry_data（問い合わせ）

社員からの要望・不具合・問い合わせを記録する表です。

> **▼ このコードがやること（先に日本語で）**：だれが、どの種類（要望/不具合/問い合わせ）で、何を聞いたか、その回答、作成・更新日時を記録します。

コピー

```
# 「問い合わせ」の表
class inquiry_data(models.Model):
  # 内容種別の選択肢
  content_list = [
    # 要望
    ('要望', '要望'),
    # 不具合報告
    ('不具合', '不具合'),
    # 質問
    ('問い合わせ' ,'問い合わせ'),
  ]

  # 問い合わせた人の番号
  employee_no2 = models.IntegerField('従業員番号')
  # 人員参照
  name = models.ForeignKey(member, verbose_name='氏名', null=True, on_delete=models.SET_NULL)
  # 種別
  content_choice = models.CharField('内容選択', choices=content_list, max_length=5)
  # 本文（長文OK）
  inquiry = models.TextField('問い合わせ')
  # 回答（未回答なら空）
  answer = models.TextField('回答', null=True, blank=True)
  # 作成日時（自動）
  created_at = models.DateTimeField(auto_now_add=True)
  # 更新日時（自動）
  updated_at = models.DateTimeField(auto_now=True)

  # 文字表現
  def __str__(self):
    # 「id＋氏名」
    return str(self.id) + str(self.name)
```

- `content_choice` … 「要望／不具合／問い合わせ」の3択。
- `inquiry` … 本文。`TextField` で長文OK。
- `answer` … 管理者が後で入れる回答。未回答なら空。
- `created_at = models.DateTimeField(auto_now_add=True)` … `auto_now_add=True` は **作成時に自動で現在日時を入れ、以後変えない。**
- `updated_at = models.DateTimeField(auto_now=True)` … `auto_now=True` は **保存のたびに現在日時へ自動更新。**回答を書き足すと更新される。

> **用語: `auto_now_add` と `auto_now` の違い** ⚠️ よく混同します。- `auto_now_add=True` … **作った瞬間だけ** 時刻を記録（＝作成日時）。- `auto_now=True` … **保存するたび** 時刻を更新（＝最終更新日時）。
> `name` は `on_delete=SET_NULL`（§5.1）なので、問い合わせた人が退職しても問い合わせ内容は残ります。

---

20. モデル⑧⑨⑩: AsyncTask / Operation_history / History

最後に、システムの裏方として働く3つの表を見ます。`AsyncTask` と `History` には「**保存時に件数上限を超えたら古いものを自動削除する**」という重要な仕掛けがあります。

20.1 AsyncTask（非同期タスク）と件数自動削除

> **▼ このコードがやること（先に日本語で）**：バックアップ等の時間のかかる処理（非同期タスク）の状態を記録します。保存のたびに件数を数え、上限1000件を超えたら古いものから自動で消します。

コピー

```
# 「非同期タスク」の表
class AsyncTask(models.Model):
  # 保持する最大件数
  MAX_RECORDS = 1000

  # タスクID（重複不可）
  task_id = models.CharField(max_length=255, unique=True)
  # 状態
  status = models.CharField(max_length=50, choices=[
    # 処理中
    ('pending', 'Pending'),
    # 成功
    ('success', 'Success'),
    # 失敗
    ('error', 'Error')
  ])
  # 結果（メッセージ等）
  result = models.TextField(null=True, blank=True)
  # 作成日時
  created_at = models.DateTimeField(auto_now_add=True)
  # 更新日時
  updated_at = models.DateTimeField(auto_now=True)

  # 文字表現
  def __str__(self):
    return f'{self.created_at} on {self.status} (TaskID: {self.task_id})'

  # ★保存処理を上書き
  def save(self, *args, **kwargs):
    # まず通常の保存を実行
    super().save(*args, **kwargs)
    # 現在の総件数を数える
    current_count = self.__class__.objects.count()

    # レコード数が許容数以上の場合の処理
    # 上限(1000)を超えていたら
    if current_count > self.MAX_RECORDS:
      # 超過レコード数分のレコード取得し削除
      # 超過件数を計算
      excess_count = current_count - self.MAX_RECORDS
      # 古い順に超過分
      oldest_records = self.__class__.objects.order_by('created_at')[:excess_count]
      # それらを
      for record in oldest_records:
        # 1件ずつ削除
        record.delete()
```

- `task_id = models.CharField(..., unique=True)` … `unique=True` で **同じIDの重複を禁止**。タスクを一意に識別。
- `status` … `pending`（処理中）/ `success`（成功）/ `error`（失敗）の3択。
- `def save(self, *args, **kwargs):` … `save` **メソッドの上書き（オーバーライド）**。Djangoが標準で持つ「保存」処理を自分の処理で包みます。
- `super().save(*args, **kwargs)` … `super()` は親クラス（`models.Model`）を指す。**まず本来の保存を実行**してから、追加処理に進みます。
- `current_count = self.__class__.objects.count()` … `self.__class__` は「このモデル自身（AsyncTask）」。`.objects.count()` で総件数を取得。
- `if current_count > self.MAX_RECORDS:` … 1000件を超えていたら、
- `oldest_records = ...order_by('created_at')[:excess_count]` … `order_by('created_at')` で **古い順** に並べ、超過した件数ぶんだけ先頭から取る。
- `for record in oldest_records: record.delete()` … それらを1件ずつ削除。

> **用語: メソッドの上書き（オーバーライド／override）** … 親クラスが持つ関数を、子クラスで同じ名前で定義し直して動作を変えること。ここでは「保存」に「古いものを掃除する」処理を足しています。
>
> **なぜ自動削除するのか:** タスクや履歴は **放っておくと無限に増え続け、データベースが肥大化** します。上限を決めて古いものから捨てることで、ディスクと性能を守る「自動ゴミ掃除」です。`AsyncTask` は1000件、`History` は50万件が上限。
>
> ⚠️ **保守の注意:** この自動削除は **`save()` が呼ばれた時だけ** 動きます。普段は問題ありませんが、「古いタスクが残っている」場合でも、新規保存が起きれば自動で掃除されます。上限値（`MAX_RECORDS`）を変えるときは、削除タイミングがこの方式だと理解した上で行ってください。

20.2 Operation_history（操作履歴）

> ▼ **このコードがやること（先に日本語で）**：だれが、どの画面で、どのモデルを、どう操作したか（結果と詳細）を記録する「操作ログ」です。

コピー

```
# 「操作履歴」の表
class Operation_history(models.Model):
  # 操作日時
  created_at = models.DateTimeField(auto_now_add=True)
  # 操作した人の番号
  employee_no4 = models.IntegerField('従業員番号')
  # 人員参照
  name = models.ForeignKey(member, verbose_name='氏名', null=True, on_delete=models.SET_NULL)
  # どの画面か
  post_page = models.CharField('ページ', max_length=255, null=True, blank=True)
  # どのモデルか
  operation_models = models.CharField('編集したモデル', max_length=255, null=True, blank=True)
  # 成功/失敗
  status = models.CharField('結果', max_length=255, null=True, blank=True)
  # 詳細メモ
  operation_detail = models.TextField('編集詳細', null=True, blank=True)

  # 文字表現
  def __str__(self):
    return str(self.id) + str(self.name) + '：' + str(self.created_at)
```

- 人がどこで何をしたかの足跡。トラブル調査で「いつ・だれが・何を変えたか」を追う一次情報になります。
- `name` はやはり `on_delete=SET_NULL`。退職者の操作履歴も氏名だけ空欄で残ります。

20.3 History（変更監査ログ）と JSONField

> ▼ **このコードがやること（先に日本語で）**：どの表のどのレコードが、追加/変更/削除されたか、その変更内容（JSON）を細かく記録する監査ログです。上限50万件で、超えたら古いものを自動削除します。

コピー

```
# 「変更監査ログ」の表
class History(models.Model):
  # 保持上限（50万件）
  MAX_RECORDS = 500000

  # 操作種別（create/update/delete等）
  operation = models.CharField(max_length=10)
  # 対象テーブル名
  table_name = models.CharField(max_length=50)
  # 対象レコードのid
  record_id = models.IntegerField()
  # 操作者のログイン番号
  login_No = models.CharField(max_length=255, blank=True, null=True)
  # ★変更内容（JSON）
  changes = jsonfield.JSONField(blank=True, null=True)
  # 記録日時
  timestamp = models.DateTimeField(auto_now_add=True)

  # 文字表現
  def __str__(self):
    return f'{self.operation} on {self.table_name} (ID: {self.record_id})'

  # ★保存処理を上書き（AsyncTaskと同じ仕組み）
  def save(self, *args, **kwargs):
    # まず通常保存
    super().save(*args, **kwargs)
    # 総件数
    current_count = self.__class__.objects.count()

    # レコード数が許容数以上の場合の処理
    # 50万件超なら
    if current_count > self.MAX_RECORDS:
      # 超過レコード数分のレコード取得し削除
      # 超過件数
      excess_count = current_count - self.MAX_RECORDS
      # 古い順に超過分
      oldest_records = self.__class__.objects.order_by('timestamp')[:excess_count]
      # それらを
      for record in oldest_records:
        # 削除
        record.delete()
```

- `changes = jsonfield.JSONField(...)` … **JSON（ジェイソン）形式** で変更内容を保存。`{"name": ["旧", "新"]}` のような「前後の値の組」を柔軟に格納できます。`CharField` と違い **構造を持ったデータ** を1フィールドに収められるのが利点。

- `operation` / `table_name` / `record_id` / `login_No` … 「どの操作を／どの表の／どのidに／だれが」行ったか。
- `save` の上書きは `AsyncTask` と同じ仕組み（並び替えキーが `timestamp` な点だけ違う）。

> **用語: JSON（ジェイソン：JavaScript Object Notation）** … `{ "キー": "値" }` の形でデータを書く、軽量で人間も読める記述形式。設定や変更履歴など、項目数が一定でないデータを保存するのに向きます。
>
> **`Operation_history` と `History` の違い:** `Operation_history` は「人の操作の足跡（画面・結果）」を人間向けに、`History` は「DB上の変更（どのレコードがどう変わったか）」を機械的・網羅的に記録します。前者は運用調査、後者は監査・復旧の材料という役割分担です。これらは `signals.py`（モデル変更の検知）から自動で書き込まれます。

---

21. 保守の心得（このデータを壊さないために）

後任者が「やってはいけないこと」「気をつけること」をまとめます。

21.1 絶対に変えてはいけないもの

対象	理由
`shop_list` の文字列（'P','W1',…）	`judgement_check`・`kosu_sort`・`break_get` の分岐条件に直結。1文字変えると工数判定が壊れる
`time_work` の文字の意味（`#`=空き, `$`=休憩, A〜x=区分）	全工数データの解釈基盤。変えると過去データが読めなくなる
休憩文字列 `#HHMMHHMM` の桁構成	`break_time_process` 等が固定位置でスライスしている
`judgement_check` の基準分数（470, 230, …）	会社の勤務ルールそのもの。シフト変更時のみ慎重に変更
`kosu_division` の「1区分=4フィールド」命名規則	`getattr(obj, f'kosu_title_{n}')` がこの規則で回している
`member` の `break_time1〜6` の命名規則	`break_get` が `tyoku` 番号からこの名前を組み立てている

21.2 変更するなら必ずマイグレーション

モデル（`models.py`）を変えたら、必ず：

コピー

```
# 変更内容から移行ファイルを作る
python manage.py makemigrations
# 実際のデータベースに反映する
python manage.py migrate
```

> ⚠️ **`makemigrations` だけで `migrate` を忘れると**、コードとデータベースの形がズレて、保存時にエラーになります。セットで実行する習慣を。

21.3 データを消す前にバックアップ

`on_delete=SET_NULL`（§5.1）のおかげで人員削除は安全側ですが、**工数記録（`Business_Time_graph`）そのものを消す操作は取り返しがつきません。**`tasks.py` のバックアップ機能を使い、消す前に必ず保存してください（次章で詳述）。

21.4「番号の冗長コピー」を理解する

`employee_no3`（工数）・`employee_no2`（問い合わせ）・`employee_no4`（操作履歴）・`employee_no5`（チーム）は、`name`（ForeignKey）が `NULL` になっても **だれの記録か追える保険** です。「ForeignKeyと番号が二重にある＝設計ミス」ではなく **意図的** だと理解してください。

21.5 件数上限の自動削除を忘れない

`AsyncTask`（1000件）・`History`（50万件）は古いものが自動で消えます。「昔のログが無い！」は **仕様** です。長期保存が必要なら別途エクスポートを。

---

22. トラブルシューティング

症状	考えられる原因	確認・対処
正しく入力したのに工数がNG（`judgement=False`）になる	`judgement_check` のショップ×直×就業形態の基準分数と、実際の `kosu_total` がズレている	§14 の表で基準値を確認。休憩(`$`)・空き(`#`)の数え方を疑う
夜勤の休憩が変な時間帯に反映される	日またぎ（`break_next_day`）の処理漏れ	§9 の `break_time_process` の戻り値が呼び出し側で2分割処理されているか確認
過去の工数の作業名が今と違って見える	工数区分定義のVerが現在のVerで解釈されている	`Business_Time_graph.def_ver2` と `kosu_division.kosu_name` が一致しているか確認
人員を削除したらエラー	通常は `SET_NULL` で安全。`PROTECT` 等に変えていないか	`on_delete` 設定（§5.1）を確認
人員を消したのに工数/問い合わせが残る	これは正常（`SET_NULL`で氏名だけ空欄になる）	バグではない。`employee_no` で追跡可能
古いタスク/履歴が消えている	`AsyncTask`(1000)・`History`(50万)の自動削除	仕様。§20.1/20.3
`time_work` の長さが288でない	書き込み処理のインデックス計算ミス、末尾`$`の余り	§13 の区切り規則と §6.3 の公式を確認
モデル変更後に保存でエラー	マイグレーション未適用	`makemigrations` → `migrate` を実行（§21.2）
作業詳細を分割すると枠数が289になる	`detail_list_summarize` で末尾に余分な`$`が付いた	§13、最後の要素に`$`を付けない規則を確認
休憩中なのに工数が入る/エラーにならない	`member.break_check` のON/OFF	§4.6・§11、本人の `break_check` 値を確認
別の直の休憩が適用されてしまう	`break_get` の `tyoku` 分岐ミス、`tyoku2` の値が不正	§4.4、`Business_Time_graph.tyoku2` の値を確認

23. 演習問題

問1. `time_work` が `max_length=288` なのはなぜですか。「1日」「5分」という言葉を使って説明してください。

問2. 休憩文字列 `#09001000` を、§7.1の表に従って「開始時・開始分・終了時・終了分」に分解し、何時から何時までの休憩か答えてください。

問3. 上の `#09001000` について、`break_time_process`（§9）に通すと `break_start`・`break_end` はいくつになりますか。§6.3の公式 `HH×12 + MM÷5` を使って計算してください。

問4. `break_time_delete`（§11）で、`member.break_check` が `True` の人と `False` の人では、休憩枠に作業が入っていたときの挙動がどう違いますか。

問5. ある社員が常昼（`tyoku='4'`）で「半前年休」を取り、残業0分でした。`judgement` が `True`（OK）になるには `kosu_total` がいくつである必要がありますか（§14.3）。

問6. `on_delete=models.SET_NULL` と `models.CASCADE` の違いを述べ、本アプリが工数記録に `SET_NULL` を選んだ理由を説明してください。

問7. `kosu_division` でVer（バージョン）管理をする理由を、「2025年のA＝段取り、2026年のA＝保全」という例を使って説明してください。

問8. `AsyncTask` の `save` メソッドが上書きされている目的は何ですか。`MAX_RECORDS` という語を使って答えてください。

問9（応用）. `detail_list_summarize`（§13）が「最後の要素にだけ `$` を付けない」のはなぜですか。読み戻す `split('$')` の動きと絡めて説明してください。

問10（応用）. `kosu_division_dictionary`（§16.3）や `break_get`（§4.4）で「フィールド名を文字列で組み立てて値を取る／選ぶ」書き方をしているのはなぜですか。フィールドが大量にある設計と絡めて説明してください。

---

24. この章のまとめ

- **モデルは「データベースの表の設計図」**。Pythonのクラスを書くだけで表ができ、`makemigrations` → `migrate` で反映する。
- 本アプリには **10のモデル** があり、**`member`（人員）が中心**。`Business_Time_graph`・`inquiry_data`・`Operation_history` が `ForeignKey` で `member` を参照する。
- `on_delete=SET_NULL` により、人員を消しても記録は氏名だけ空欄で残る。`employee_no` の冗長コピーが追跡の保険になる。
- **最重要①: `time_work` は288枠・5分刻み設計**。1日を288コマに分け、各コマを1文字（A〜x=区分, `#`=空き, `$`=休憩）で表す。インデックス公式は `HH×12 + MM÷5`。
- **最重要②: 休憩は `#HHMMHHMM` の9文字**。`#` を飛ばして時・分を固定位置でスライスし、288枠インデックスに変換する。`break_time_process` が日またぎ（夜勤）も判定する。
- `kosu_utils.py` の各関数は、この288枠を **書き込み**（`kosu_write`）／**休憩消去**（`break_time_delete`）／**休憩確定**（`break_time_write`）／**文字列化**（`detail_list_summarize`）／**整合性判定**（`judgement_check`）する部品。

- `judgement_check` **は会社の勤務ルールの塊。**470・230・240… などの数字は所定労働時間そのもので、ショップ×直×就業形態ごとに基準が違う。
- `member` **の休憩は「直 × 4種」で24フィールド。**`break_get` が `tyoku` 番号からその直の4つを選ぶ。
- `kosu_division` **は最大50区分 × Ver管理。**Verを分けることで過去の工数の意味を壊さずに定義を更新できる。
- `AsyncTask` **(1000件)・**`History` **(50万件)** は `save` の上書きで、上限を超えた古いデータを自動削除する。
- 保守の鉄則：**ショップ名・**`time_work` **の文字・休憩の桁構成・判定の基準分数・各種フィールドの命名規則を勝手に変えない。**変えるなら必ずマイグレーションとバックアップを。

次の章では、ここまで理解したデータ構造とコードを、実際の環境で **動かし・守り・配る** 方法（運用・保守・デプロイ）を学びます。

→ 次章: 09_運用_保守_デプロイ.md